Example Project Inspiration: ave.ai

Important Note

This example is only meant to help you understand how to derive business needs from a real website. You should not use this example for your project.

Your goal is to observe the type of business it operates, identify possible user needs, and design your own simplified backend MVP based on a small part of that business.

Company Example: ave.ai

ave.ai is a financial-domain platform focused on crypto trading, token analytics, blockchain data, and market monitoring.

A platform like ave.ai helps users discover tokens, monitor market activity, evaluate trading opportunities, and react to fast-changing financial data.

Business Domain

Financial Technology / Crypto Market Analytics

Business Need 1: Smart Alerts

Business Need

Users need to be notified when important market events happen instead of manually checking token pages over and over again.

In crypto markets, meaningful events can happen within minutes. A user may miss a sudden price drop, volume spike, or liquidity change if they are not actively watching the platform.

User Problems

Users may miss important events such as:

- Sudden price drops
- Sudden price increases
- Large volume spikes
- Liquidity decreases
- Possible rug-pull risk signals
- Whale-related activity
- Unusual token behavior

Business Goal

The platform wants to become a proactive decision-support tool.

Instead of only showing data when users visit the site, the system should help users notice important changes automatically.

Example User Stories

- As a user, I want to create an alert when a token price drops by more than 5%.
- As a user, I want to receive a notification when trading volume increases significantly.
- As a user, I want to monitor liquidity changes for tokens I care about.
- As a user, I want to view my alert history so I can understand what happened.

Example Business Endpoints

These are example endpoints:

- `POST /alerts`
  - Create a new token alert rule.
- `GET /alerts`
  - View all alert rules for the authenticated user.
- `GET /alerts/{alert_id}`
  - View details of one alert rule.
- `PATCH /alerts/{alert_id}`
  - Update an existing alert rule.
- `DELETE /alerts/{alert_id}`
  - Delete an alert rule.
- `GET /notifications`
  - View triggered alert notifications.

- `PATCH /notifications/{notification_id}/read`
    - Mark a notification as read.

Business Value

Smart alerts make the product more useful because users do not need to constantly monitor the platform manually.

This can improve:

- User retention
- User trust
- User engagement
- Paid subscription conversion

Business Need 2: Paid Feature Access and Entitlements

Business Need

The business needs to control which users can access free features versus paid features.

In a financial platform, some features may be more valuable and should only be available to paid users. For example, advanced alerts, real-time data, higher usage limits, or deeper historical analysis may be part of a premium plan.

If feature access is only controlled by hiding buttons on the frontend, users may still try to call backend APIs directly. Therefore, access control should be enforced on the backend.

User Problems

From the business perspective, users may:

- Access premium features without paying
- Make too many API requests
- Scrape large amounts of financial data
- Bypass frontend restrictions
- Use advanced features without the correct subscription plan

Business Goal

The platform wants to protect premium features and create a clear difference between free and paid plans.

Example User Stories

- As a free user, I can create a limited number of alerts.
- As a paid user, I can create more alerts.
- As a free user, I may only access basic token data.
- As a paid user, I can access advanced alert types or longer historical comparisons.
- As a business, we want to prevent unpaid users from accessing premium APIs.

Example Business Endpoints

These are example endpoints:

- `GET /me/plan`
  - View the authenticated user's current subscription plan.
- `GET /me/entitlements`
  - View which features the authenticated user can access.
- `POST /subscriptions`
  - Create or simulate a user subscription.
- `PATCH /subscriptions/{subscription_id}`
  - Update a user's subscription plan.
- `GET /features`
  - View available platform features and plan requirements.

Business Value

Entitlements support monetization.

This feature helps the business:

- Protect paid features
- Encourage upgrades
- Reduce abuse
- Control usage limits
- Create a sustainable subscription model

Business Need 3: Saved Views

Business Need

Power users need a way to save and reuse their preferred filters, searches, and analysis setups.

Without saved views, users may have to rebuild the same token filters every time they use the platform.

For example, a user may repeatedly search for:

- New tokens
- High-volume tokens
- Low market cap tokens
- Tokens with liquidity risk
- Tokens with unusual market activity

This repeated setup wastes time and can make the product frustrating to use.

Business Goal

The platform wants to make user workflows faster, more personalized, and more repeatable.

Example User Stories

- As a user, I want to save a group of token filters as a named view.
- As a user, I want to quickly reopen a saved view later.
- As a user, I want to delete saved views I no longer use.
- As a user, I want to apply a saved view to get updated token results.
- As a paid user, I may be allowed to create more saved views.

Example Saved Views

Possible saved view names:

- “New High Volume Tokens”
- “Small Cap Momentum”
- “Liquidity Risk Watch”
- “Potential Whale Activity”
- “High Volume Low Liquidity”

Example Business Endpoints

These are example endpoints:

- `POST /saved-views`
  - Create a new saved view.
- `GET /saved-views`
  - View all saved views for the authenticated user.
- `GET /saved-views/{view_id}`
  - View one saved view.
- `PATCH /saved-views/{view_id}`
  - Update a saved view.
- `DELETE /saved-views/{view_id}`
  - Delete a saved view.
- `GET /saved-views/{view_id}/results`
  - Apply a saved view and return matching mock token results.

Business Value

Saved views improve user productivity and retention.

This feature helps make the product “sticky” because users build personalized workflows inside the platform. Once users save their workflows, they have a stronger reason to return.

Business Need 4: Historical Comparison

Business Need

Users need context, not just current numbers.

Current token data alone may not be enough for decision-making. Users may need to compare current metrics against previous values to understand whether a change is meaningful.

For example, a token’s current volume may look high, but the user needs to know whether it is high compared with yesterday or last week.

User Problems

Users may ask:

- Is this token's volume higher than usual?
- Has liquidity decreased compared with yesterday?
- Did price increase while liquidity dropped?
- Did market cap grow in a healthy way?
- Is this movement normal or unusual?
- Is this token becoming riskier over time?

Business Goal

The platform wants to help users interpret market movement, not just display raw data.

Example User Stories

- As a user, I want to compare a token's current price with its price 24 hours ago.
- As a user, I want to compare current liquidity with liquidity 7 days ago.
- As a user, I want to see percentage changes for price, volume, liquidity, and market cap.
- As a paid user, I may be able to compare longer time windows.

Example Business Endpoints

These are example endpoints:

- `GET /tokens/{token_id}/metrics`
  - View current mock metrics for a token.
- `GET /tokens/{token_id}/compare?baseline=24h`
  - Compare current token metrics with a previous time period.
- `GET /tokens/{token_id}/snapshots`
  - View historical mock snapshots for a token.
- `POST /tokens/{token_id}/snapshots`
  - Create mock historical snapshot data.
- `GET /tokens/{token_id}/risk-summary`
  - Return a simple risk summary based on metric changes.

Business Value

Historical comparison helps users make better financial decisions.

This feature can improve:

- User trust

- Product value
- Analytical depth
- Paid subscription conversion
- User confidence when interpreting market changes

Possible Database Models

A simplified version of this project could include the following models.

User

Stores authentication-related user information.

Possible fields:

- `id`
- `email`
- `hashed_password`
- `created_at`

SubscriptionPlan

Stores plan-level information.

Possible fields:

- `id`
- `name`
- `max_alerts`
- `max_saved_views`
- `real_time_data_access`
- `historical_window_limit`

Token

Stores mock token information.

Possible fields:

- `id`
- `chain`
- `contract_address`
- `symbol`
- `name`
- `created_at`

TokenMetricSnapshot

Stores mock token market data at a specific point in time.

Possible fields:

- `id`
- `token_id`
- `price`
- `volume`
- `liquidity`
- `market_cap`
- `snapshot_time`

AlertRule

Stores alert rules created by users.

Possible fields:

- `id`
- `user_id`
- `token_id`
- `condition_type`
- `threshold`
- `direction`

- `time_window`
- `is_active`
- `created_at`

Notification

Stores notifications created when an alert is triggered.

Possible fields:

- `id`
- `user_id`
- `alert_rule_id`
- `message`
- `is_read`
- `created_at`

SavedView

Stores user-created token research views.

Possible fields:

- `id`
- `user_id`
- `name`
- `filter_config`
- `created_at`
- `updated_at`

Key Points

When looking at a real website like ave.ai, ask:

- What problem is this business solving?

- Who are the users?
- What information do users need?
- What actions do users need to perform?
- What data needs to exist in the backend?
- What API endpoints would support those actions?
- Which features create business value?
- Which features could be free vs. paid?
- What privacy, security, or access-control concerns exist?